

# **GPU Accelerated Material Point Method for Advection-Diffusion Problems**

Paolo Lo Iacono  
PhD Student  
Department of Mathematics  
Politecnico di Milano

1. Governing Equations
2. Discretization
3. Implementation
4. Results



# Governing Equations

## Advection-Diffusion Equation for Quantity $u$ :

$$\frac{\partial u}{\partial t} + \nabla \cdot \mathbf{J} = 0$$



$$\mathbf{J} = \mathbf{J}^{drift} + \mathbf{J}^{diff} = \beta u - D \nabla u$$

# Grid functions and particle functions

We partition the domain into a uniform cartesian grid  $T_h$  and represent/discretize the function  $u(x)$  by assigning its values to a set of "particles" located at the points  $x_p$

$$U = \sum_p U_p \delta(x - x_p)$$

The field  $U$  defined at the particle locations can be "transferred" to the grid as follows:

First, we define the continuous piecewise bilinear Lagrangian basis functions  $\phi_i$  for the finite element space  $Q_1(T_h)$  over  $T_h$ , and then perform a Galerkin projection of  $U$  onto  $Q_1(T_h)$ .

This operation is referred to as the "particle-to-grid" transfer (P2G).

$$M_i u_i = \sum_j u_j \int_{\Omega} \varphi_j(x) \varphi_i(x) =$$

$$\int_{\Omega} u(x) \varphi_i(x) = \sum_p \int_{\Omega} U_p \delta(x - x_p) \varphi_i(x) = \sum_p U_p \varphi_i(x_p) \quad \forall i$$

P2G

# Transfer of the Convective Flux

To transfer vector fields such as the flux  $\mathbf{J}$  we apply the **P2G** procedure to each cartesian component

$$J_x^{\text{drift}} = \mathbf{J}^{\text{drift}} \cdot \hat{x}$$

$$J_y^{\text{drift}} = \mathbf{J}^{\text{drift}} \cdot \hat{y}$$

**P2G:**

$$M_i \mathbf{j}_i^{\text{drift}} \cdot \hat{x} =$$

$$\sum_k \int_{\Omega} \mathbf{j}_k^{\text{drift}} \cdot \hat{x} \varphi_k(x) \varphi_i(x) = \int_{\Omega} \mathbf{J}^{\text{drift}} \cdot \hat{x} \varphi_i(x) =$$

$$\sum_p \int_{\Omega} \beta \cdot \hat{x} U_p \delta(x - x_p) \varphi_i(x) =$$

$$\sum_p \beta(x_p) \cdot \hat{x} U_p \varphi_i(x_p) \quad \forall i$$

Here we are applying mass lumping

# Transfer of the Diffusive Flux

The transfer of the diffusive flux involves an additional integration by parts this procedure is called P2GD

$$\mathbf{J}^{\text{diff}} = -D \nabla u$$

**P2GD:**

$$\begin{aligned} M_i \mathbf{j}_i^{\text{diff}} \cdot \hat{x} &= \\ \sum_k \int_{\Omega} \mathbf{j}_k^{\text{diff}} \cdot \hat{x} \varphi_k(x) \varphi_i(x) &= \int_{\Omega} \mathbf{J}^{\text{diff}} \cdot \hat{x} \varphi_i(x) = \\ - \int_{\Omega} D \partial_x u \varphi_i(x) &= \\ \int_{\Omega} D u \partial_x \varphi_i(x) - \int_{\partial\Omega} D \partial_x u \varphi_i(x) &= \\ \sum_p \int_{\Omega} D U_p \delta(x - x_p) \partial_x \varphi_i(x) &= \\ \sum_p D U_p \partial_x \varphi_i(x_p) \quad \forall i \end{aligned}$$

# Velocity Update

We compute:

$$\mathbf{v}_i^{\text{drift}} = \frac{M_i \mathbf{j}_i^{\text{drift}}}{M_i u_i} \quad \mathbf{v}_i^{\text{diff}} = \frac{M_i \mathbf{j}_i^{\text{diff}}}{M_i u_i}$$

Then transfer back to the particles doing a **G2P** operation

$$\mathbf{V}_p = \sum_i \left( \mathbf{v}_i^{\text{drift}} + \mathbf{v}_i^{\text{diff}} \right) \varphi_i(x_p) \quad \forall p$$

# Algorithm

While  $t < t_{\text{final}}$

1. Grid Reset
2. **G2P** of  $\beta_x, \beta_y$
3. **P2G** of  $u, J_{\text{drift}_x}, J_{\text{drift}_y}$
4. **P2GD** of  $J_{\text{diff}_x}, J_{\text{diff}_y}$
5. Apply Boundary Conditions
6. Compute Nodal Velocity:  $V = (J_{\text{drift}} + J_{\text{diff}})/u$
7. **G2P** of  $V_x, V_y$
8. Compute  $dt$  and advance time ( $t = t + dt$ )
9. Move particles and update connectivity



# Implementation

To enable the use of GPUs we rely on the NVIDIA Thrust library, a C++ template library for CUDA .

```
thrust::for_each(execution_policy, first, last, kernel);
```

**execution\_policy** Specifies where and how the operation runs (e.g., `thrust::host`, `thrust::device`).

**first, last** The range of iterators defining the elements to process.

**kernel** A functor or lambda expression executed on each element.

# Implementation

Thrust provides two primary data structures:

`thrust::host_vector`: Allocates and manages memory on the host (CPU).

`thrust::device_vector`: Allocates and manages memory on the device (GPU).

Both containers provide an interface identical to `std::vector`, allowing familiar methods like `.push_back()`, `.size()`, `.resize()`, and iterator-based access.

Assigning a `host_vector` to a `device_vector` (or vice versa) automatically triggers PCIe memory transfers (`cudaMemcpy`).

GPU memory is automatically allocated upon creation and deallocated when the vector goes out of scope.

# Implementation

In quadgrid all functions/functors are decorated with HOST or DEVICE (or both).

HOST and DEVICE are macros for the CUDA `__host__` and `__device__` execution space qualifiers that tell the compiler where a function should be compiled and where it can be executed.



# Implementation - G2P Class

```
template<typename GVAR_t, typename PVAR_t, typename P2C_t, typename GRIDM_t>
class
g2p_helper_t {

    using idx_t = particles_t::idx_t;
    PVAR_t x;
    PVAR_t y;
    const GRIDM_t M;
    const GVAR_t gvar;
    const P2C_t ptcl_to_grd;
    const idx_t nrows;
    const real_t hx;
    const real_t hy;
    PVAR_t dprop;
    bool apply_mass;

public :
    g2p_helper_t (const PVAR_t x_, const PVAR_t y_, GRIDM_t M_,
        const GVAR_t gvar_, const P2C_t ptcl_to_grd_, const idx_t nrows_,
        const real_t hx_, const real_t hy_, PVAR_t dprop_, bool apply_mass_)
        : x(x_), y(y_), M(M_), gvar(gvar_),
        ptcl_to_grd(ptcl_to_grd_), nrows(nrows_), hx(hx_), hy(hy_),
        dprop(dprop_), apply_mass(apply_mass_) {};
```



# Implementation - G2P Kernel

## DEVICE

```
void
operator() (idx_t ip) {
    using qgt = quadgrid_t<GVAR_t>;
    real_t N = 0.0;
    auto xx = x[ip];
    auto yy = y[ip];
    auto r = qgt::gind2row (ptcl_to_grd[ip], nrows);
    auto c = qgt::gind2col (ptcl_to_grd[ip], nrows);
    dprop[ip] = 0.;
    for (idx_t inode = 0; inode < 4; ++inode) {
        N = apply_mass ? qgt::shp (xx, yy, inode, c, r, hx, hy) * M[qgt::gt(inode, c, r, nrows)] :
            qgt::shp (xx, yy, inode, c, r, hx, hy);
        dprop[ip] += N * gvar[qgt::gt(inode, c, r, nrows)];
    }
}
};
```

# Implementation - P2G Kernel

```
DEVICE
void
operator() (idx_t ip) {
    using qgt = quadgrid_t<GVAR_t>;
    real_t N = 0.0;
    auto xx = x[ip];
    auto yy = y[ip];
    auto r = qgt::gind2row (ptcl_to_grd[ip], nrows);
    auto c = qgt::gind2col (ptcl_to_grd[ip], nrows);
    for (idx_t inode = 0; inode < 4; ++inode) {
        N = apply_mass ? qgt::shp (xx, yy, inode, c, r, hx, hy)/M[qgt::gt(inode, c, r, nrows)] :
            qgt::shp (xx, yy, inode, c, r, hx, hy);
        atomicAdd(&(gvar[qgt::gt(inode, c, r, nrows)]), N*dprop[ip]);
    }
}
```

# Implementation - P2GD Kernel

```
DEVICE
void
operator() (idx_t ip) {
    using qgt = quadgrid_t<GVAR_t>;
    real_t Nx = 0.0, Ny = 0.0;
    auto xx = x[ip];
    auto yy = y[ip];
    auto r = qgt::gind2row (ptcl_to_grd[ip], nrows);
    auto c = qgt::gind2col (ptcl_to_grd[ip], nrows);
    for (idx_t inode = 0; inode < 4; ++inode) {
        Nx = apply_mass ? qgt::shg (xx, yy, 0, inode, c, r, hx, hy) / M[qgt::gt(inode, c, r, nrows)] :
            qgt::shg (xx, yy, 0, inode, c, r, hx, hy);
        Ny = apply_mass ? qgt::shg (xx, yy, 1, inode, c, r, hx, hy) / M[qgt::gt(inode, c, r, nrows)] :
            qgt::shg (xx, yy, 1, inode, c, r, hx, hy);

        atomicAdd(&(gvar[qgt::gt(inode, c, r, nrows)]), (Nx*dpropx[ip] + Ny*dpropy[ip])*dproparea[ip]);
    }
}
```

# Implementation - Custom G2P Kernel - Total Velocity

We need the total velocity:  $V = J_{\text{drift}}/m + J_{\text{diff}}/m$

```
DEVICE
void
operator() (idx_t ip) {
    using qgt = quadgrid_t<GVAR_t>;
    real_t N = 0.0;
    auto xx = x[ip];
    auto yy = y[ip];
    auto r = qgt::gind2row (ptcl_to_grd[ip], nrows);
    auto c = qgt::gind2col (ptcl_to_grd[ip], nrows);
    for (idx_t inode = 0; inode < 4; ++inode) {

        N = apply_mass ? qgt::shp (xx, yy, inode, c, r, hx, hy) * M[qgt::gt(inode, c, r, nrows)] :
            qgt::shp (xx, yy, inode, c, r, hx, hy);

        dprop[ip] += N * (gvar1[qgt::gt(inode, c, r, nrows)] + gvar2[qgt::gt(inode, c, r, nrows)]);
    }
};
```

# Implementation - G2PD Kernel

```
DEVICE
void
operator() (idx_t ip) {
    using qgt = quadgrid_t<GVAR_t>;
    real_t Nx = 0.0, Ny = 0.0;
    auto xx = x[ip];
    auto yy = y[ip];
    auto r = qgt::gind2row (ptcl_to_grd[ip], nrows);
    auto c = qgt::gind2col (ptcl_to_grd[ip], nrows);

    for (idx_t inode = 0; inode < 4; ++inode) {
        Nx = apply_mass ?
        qgt::shg (xx, yy, 0, inode, c, r, hx, hy) * M[qgt::gt(inode, c, r, nrows)] :
        qgt::shg (xx, yy, 0, inode, c, r, hx, hy);

        Ny = apply_mass ?
        qgt::shg (xx, yy, 1, inode, c, r, hx, hy) * M[qgt::gt(inode, c, r, nrows)] :
        qgt::shg (xx, yy, 1, inode, c, r, hx, hy);
        dpropx[ip] += Nx * gvar[qgt::gt(inode, c, r, nrows)];
        dpropy[ip] += Ny * gvar[qgt::gt(inode, c, r, nrows)];
    }
} };
```



# Taylor-Aris Dispersion

Taylor–Aris dispersion describes the enhanced spreading of solute in a fluid due to the coupled effects of advection and diffusion.

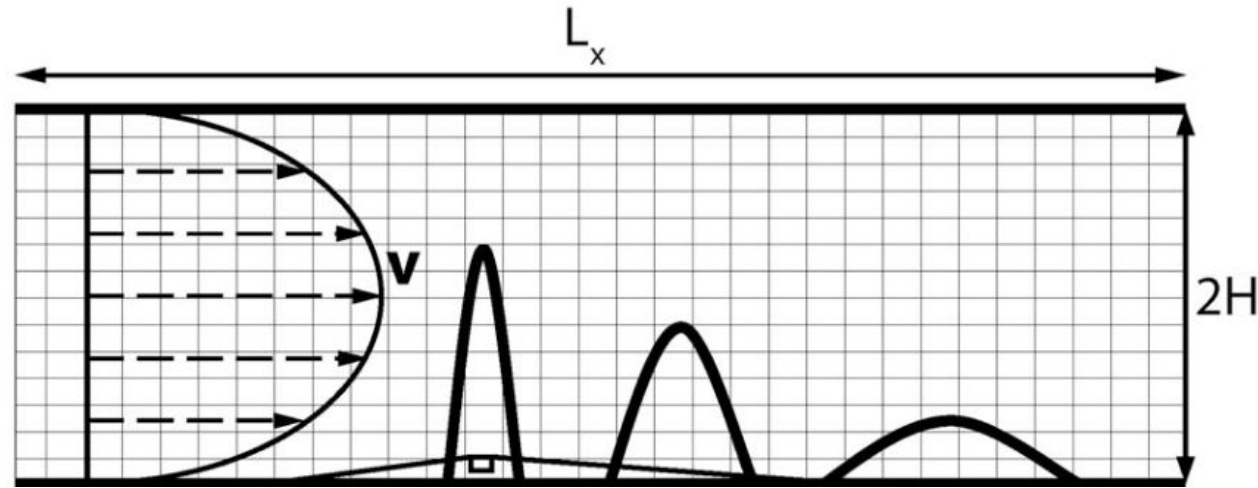


Fig 1: Two dimensional rectangular domain discretized with a cartesian grid ( $h_x = 0.01$ ,  $h_y=0.002$ ).

$$v(y) = \frac{3}{2}v_{\text{avg}}\left(1 - \frac{y^2}{H^2}\right)$$

$$V_{\text{avg}} = 1000$$

$$H = 0.1$$

$$D=1$$



$$Pe = 100$$

Advection Dominated

# How to compile

You can compile the code on three different backends: NVIDIA GPU, AMD GPU, CPU (Thrust).

NVIDIA GPU: the command *make taylor\_dispersion* runs:

```
nvcc $(CXXFLAGS) $(CPPFLAGS) -arch=sm_80 --expt-relaxed-constexpr -DUSE_THRUST -x cu  
$(QUADGRIDSRC) $(TIMERSRC) taylor_dispersion.cpp -o taylordispersion
```

-arch=sm\_80 : sets the cuda compute capability

--expt-relaxed-constexpr : allows the device to call host constexpr functions

-DUSE\_THRUST : if used it includes all the needed thrust headers

-x cu : tells the compiler to read the .cpp main file as a .cu file



# How to compile

You can compile the code on three different backends: NVIDIA GPU, AMD GPU, CPU (Thrust).

AMD GPU: the command *make hip* runs:

```
hipcc $(CXXFLAGS) $(CPPFLAGS) -DUSE_THRUST $(QUADGRIDSRC) $(TIMERSRC) taylor_dispersion.cpp -o  
taylordispersion.hip
```



# How to compile

You can compile the code on three different backends: NVIDIA GPU, AMD GPU, CPU (Thrust).

CPU: the command *make thrust\_cpu* runs:

```
g++ $(THRUST_CPU_CXXFLAGS) $(CPPFLAGS) -DUSE_THRUST  
-DTHRUST_DEVICE_SYSTEM=THRUST_DEVICE_SYSTEM_CPP $(QUADGRIDSRC) $(TIMERSRC)  
taylor_dispersion.cpp -o taylordispersion.cpu
```

-DTHRUST\_DEVICE\_SYSTEM=THRUST\_DEVICE\_SYSTEM\_CPP : tells the compiler to use device=cpu

-DTHRUST\_CPU: includes atomicAdd.h

# Taylor-Aris Dispersion



# Results

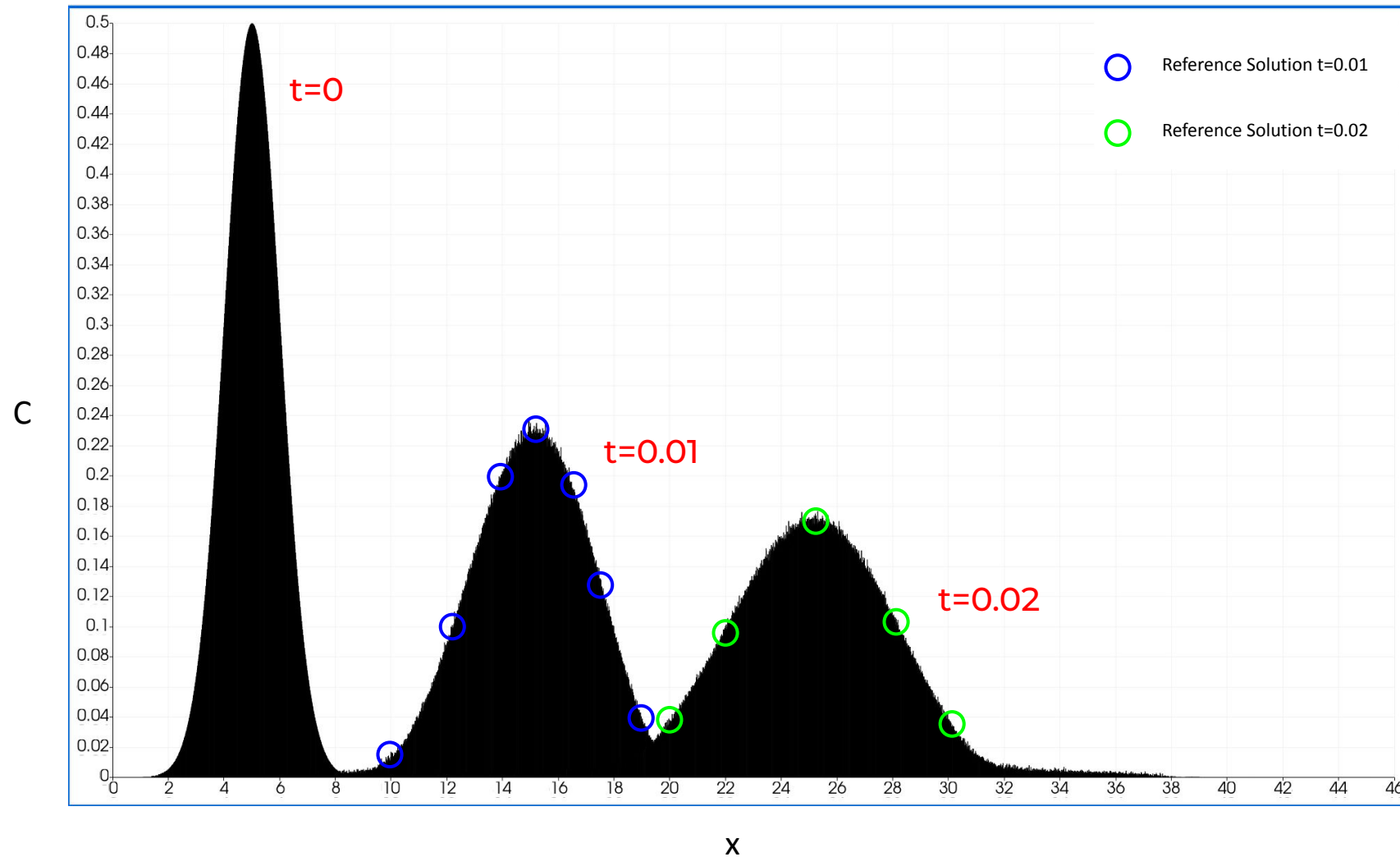


Fig 2: Temporal evolution of the cross sectional average of the concentration profile.

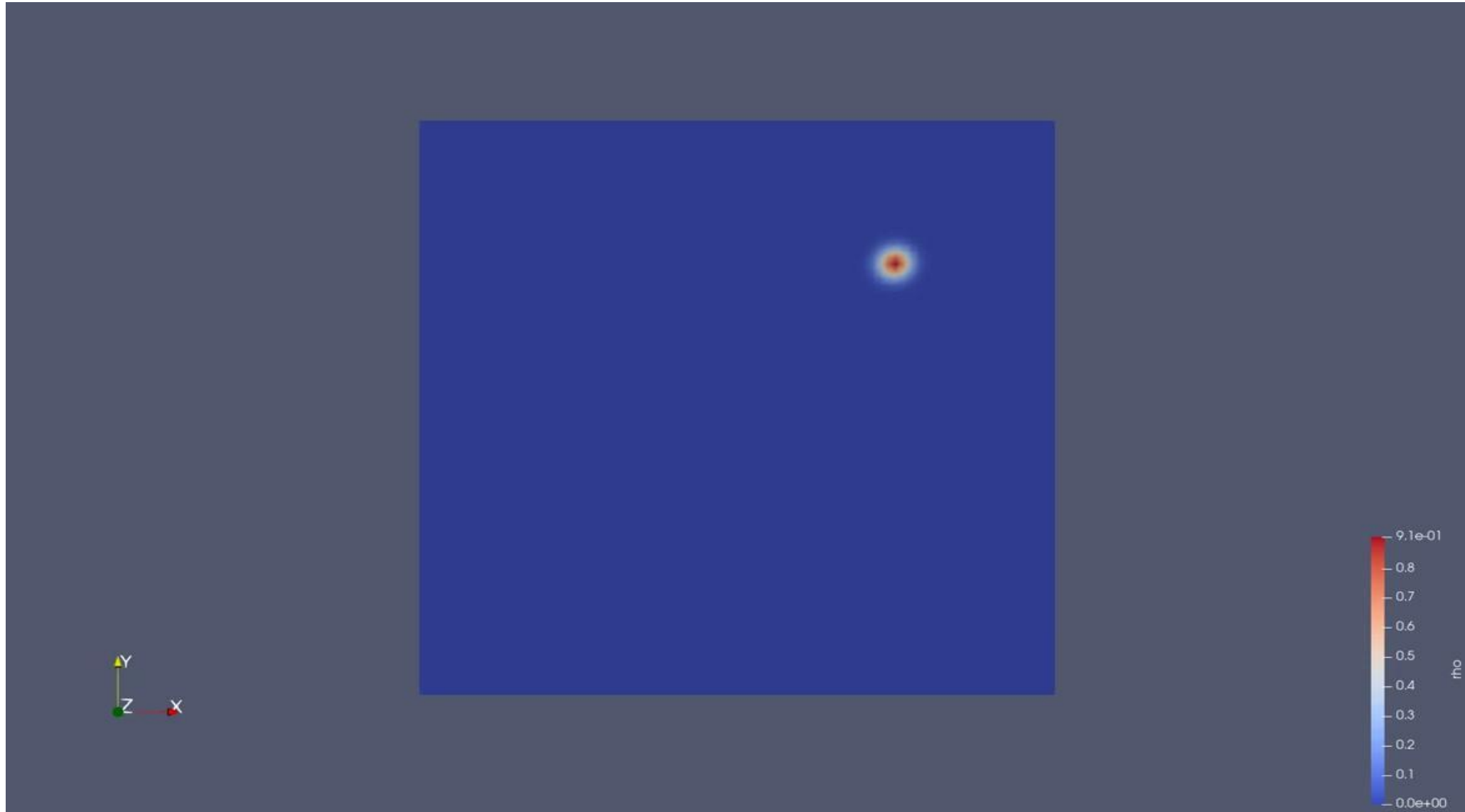
# Performances

---

- CPU: Total Measured Wall Clock Time: 1151.09 s
- GPU: Total Measured Wall Clock Time: 28.0888 s

Around 40x times faster!

# Circling ball



---

**Thank you for your attention!**